جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

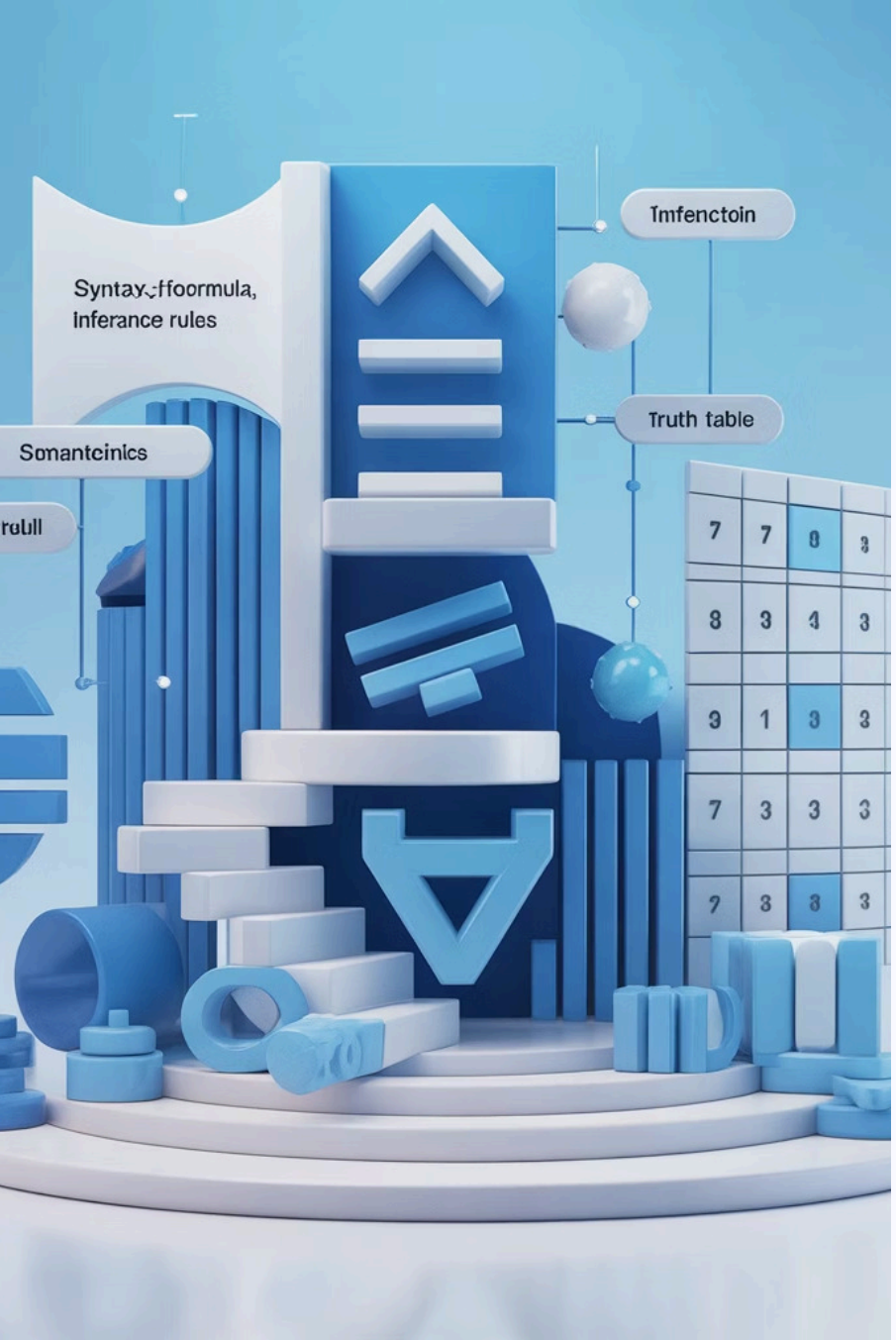
Lecture 7.4: Propositional Logic Part 3: Resolution and Conjunctive Normal Form

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture delves into the foundations of logical reasoning in AI through an introduction to Propositional Logic. You'll learn how AI agents utilize this formal logic to represent knowledge and make informed decisions.





Building Blocks of Logic

Syntax

Defines the set of valid formulas in our logical language

Example: Rain ∧ Wet

Semantics

For each formula, specifies a set of models (assignments or world configurations)

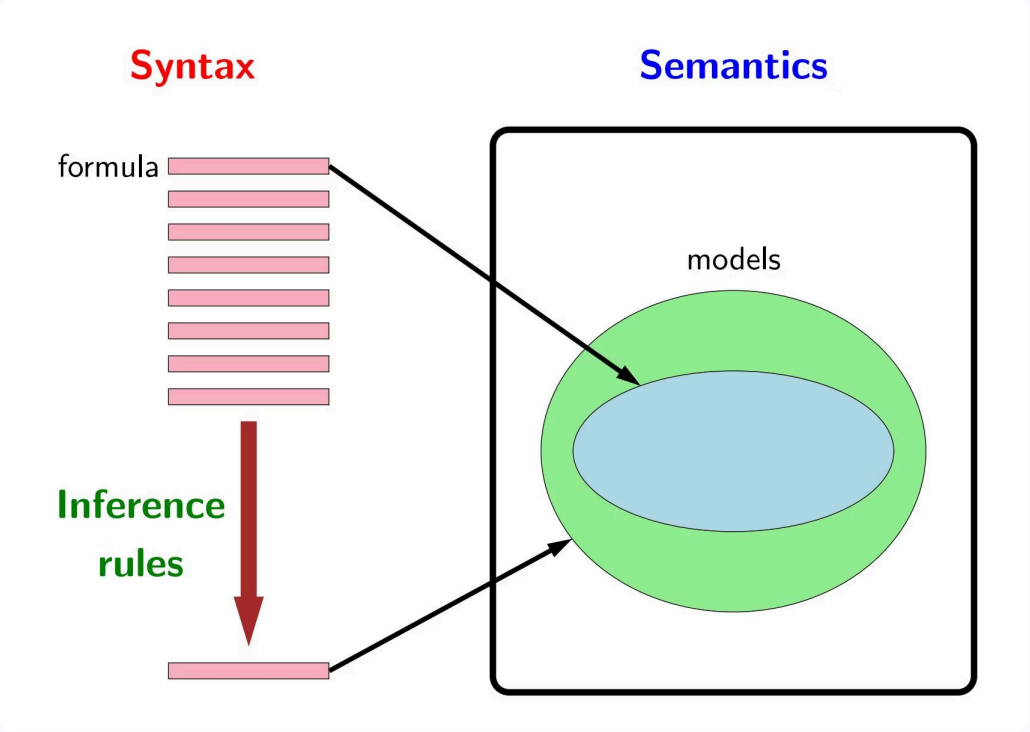
Assignments of truth values

Inference Rules

Given formula f, determines what new formulas g can be derived with certainty

Example: Rain ∧ Wet → Rain

Formal Logic Landscape



The Connection

Logic provides a formal language to describe the world

Formulas live in syntax-land

Models represent possible configurations in semantics-land

Interpretation function connects them: M(f)

The Inference Task

Input Components

Knowledge Base (KB): A set of formulas known to be true

Example: {Rain, Rain → Wet}

Query Formula (f): What we want to verify

Example: Wet

Output

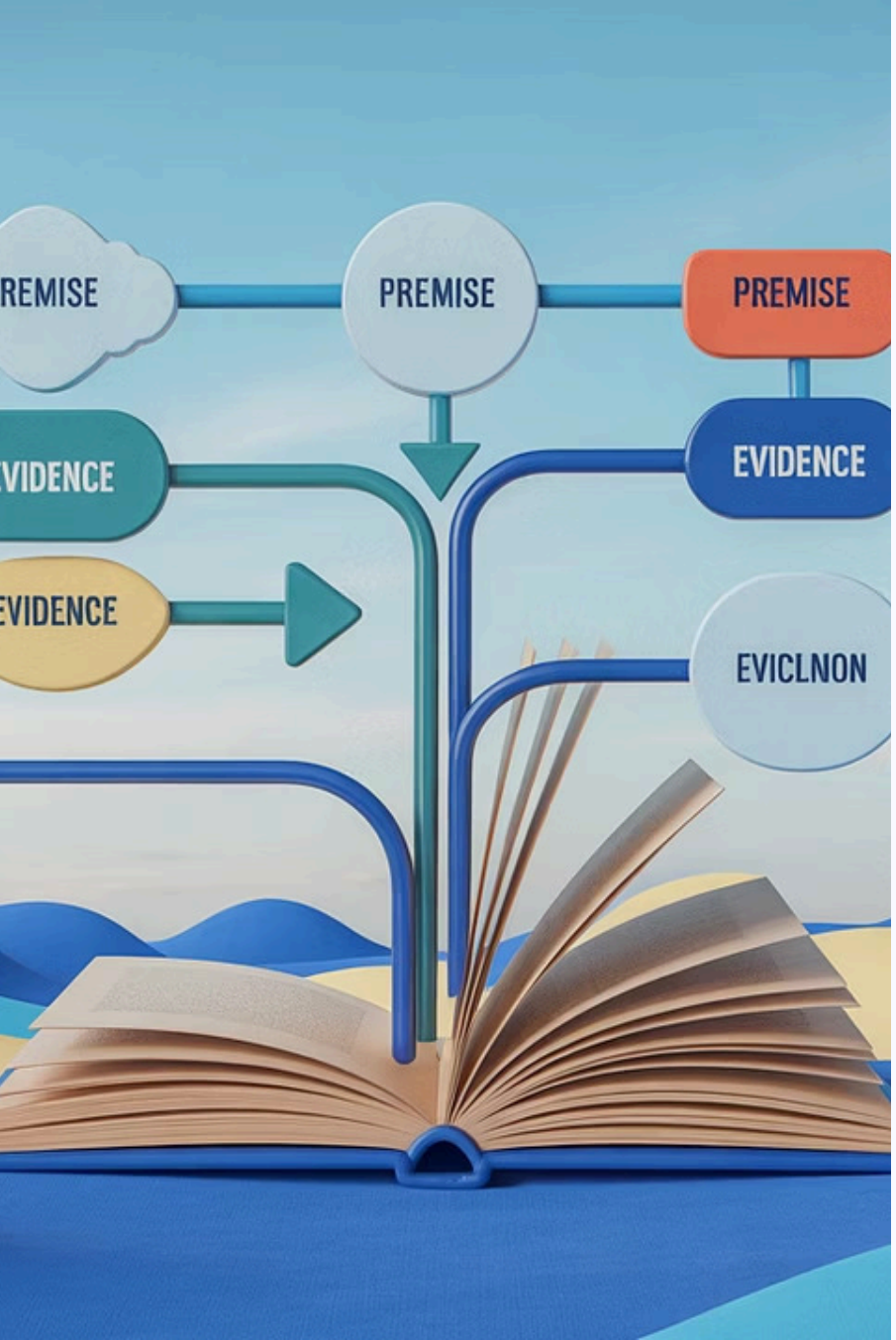
Whether KB entails f (KB ▯ f)?

Answer: yes

Formal Definition:

$$KB \models f \iff \mathcal{M}(f) \supseteq \mathcal{M}(KB)$$

The knowledge base represents the conjunction of all formulas we know to be true. The central goal is determining whether a query formula is entailed by, contradictory with, or contingent on the KB.

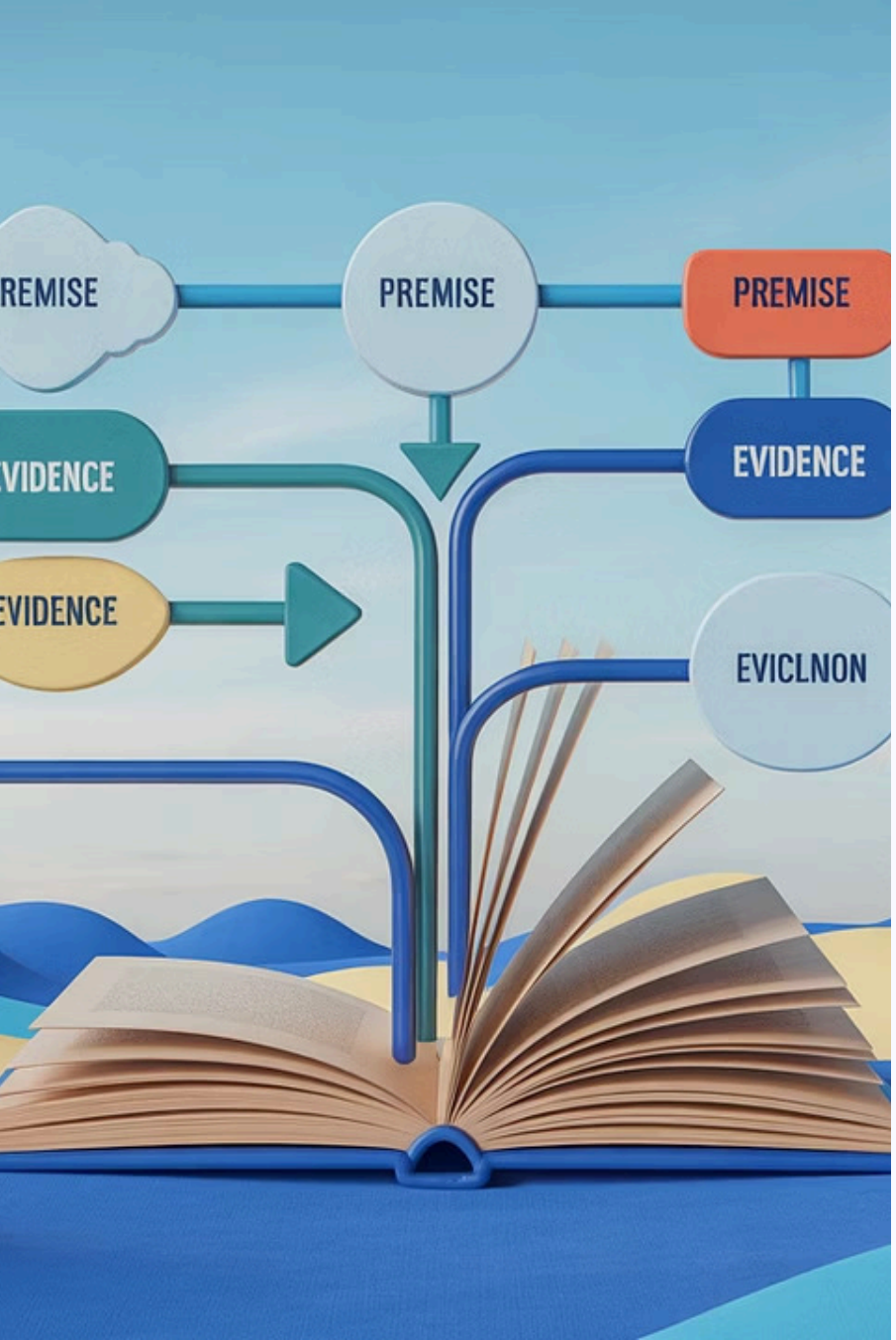


Inference Algorithms

Apply **inference rules** repeatedly to derive new knowledge

Modus Ponens Rule

$$\frac{p_1, \cdots, p_k, (p_1 \wedge \cdots \wedge p_k) \rightarrow q}{q}$$



Inference Algorithms

Apply **inference rules** repeatedly to derive new knowledge

Modus Ponens Rule

$$\frac{p_1, \cdots, p_k, (p_1 \wedge \cdots \wedge p_k) \rightarrow q}{q}$$

Key Desired Properties

- **Soundness:** Never derive false formulas
- **Completeness:** Derive all true formulas

Soundness is easy to verify, **completeness** is more challenging.

Common Inference Rules in Propositional Logic (1)

1

Modus Ponens ($\rightarrow$ Elimination)

$$\frac{P, P \rightarrow Q}{Q}$$

Example: “It’s raining” and “If it rains, it’s wet” $\rightarrow$ “It’s wet.”

2

Modus Tollens

$$\frac{\neg Q, P \rightarrow Q}{\neg P}$$

Example: “It’s not wet” and “If it rains, it’s wet” $\rightarrow$ “It’s not raining.”

3

Hypothetical Syllogism

$$\frac{P \rightarrow Q, Q \rightarrow R}{P \rightarrow R}$$

Example: “If it rains, it gets wet” and “If it’s wet, it’s slippery” $\rightarrow$ “If it rains, it’s slippery.”

4

Disjunctive Syllogism

$$\frac{P \vee Q, \neg P}{Q}$$

Example: “It’s raining or cloudy” and “It’s not raining” $\rightarrow$ “It’s cloudy.”

Common Inference Rules in Propositional Logic (2)

1

Simplification ($\wedge$ Elimination)

$$\frac{P \wedge Q}{P}$$

Example: “It’s raining and cold” → “It’s raining.”

2

Conjunction ($\wedge$ Introduction)

$$\frac{P, Q}{P \wedge Q}$$

Example: “It’s raining.” “It’s cold.” → “It’s raining and cold.”

3

Addition ($\vee$ Introduction)

$$\frac{P}{P \vee Q}$$

Example: “It’s raining” → “It’s raining or sunny.”

4

Resolution

$$\frac{P \vee Q, \neg P \vee R}{Q \vee R}$$

Example: “It’s raining or cloudy” and “It’s not raining or cold” → “It’s cloudy or cold.”

(Used heavily in **automated reasoning**.)

💡 Inference Rules - Summary Table

Rule	Pattern	Result
Modus Ponens	$P, P \to Q \implies Q$	Apply conditional ✅
Modus Tollens	$\neg Q, P \to Q \implies \neg P$	Negate implication ↩️
Hypothetical Syllogism	$P \to Q, Q \to R \implies P \to R$	Chain implications 🔗
Disjunctive Syllogism	$P \lor Q, \neg P \implies Q$	Eliminate false part ✂️
Simplification	$P \land Q \implies P$	Drop one 📉
Conjunction	$P, Q \implies P \land Q$	Combine ➕
Addition	$P \implies P \lor Q$	Add disjunction 🎁
Resolution	$P \lor Q, \neg P \lor R \implies Q \lor R$	Combine disjunctions ✨

Formula Types in Propositional Logic

General Propositional Logic

Any legal combination of symbols:

$$(\text{Rain} \wedge \text{Snow}) \rightarrow (\text{Traffic} \vee \text{Peaceful}) \wedge \text{Wet}$$

Highly expressive but requires more powerful inference rules **(computation intensive)**.

Horn Clauses Only

Restricted form:

$$(\text{Rain} \wedge \text{Snow}) \rightarrow \text{Traffic}$$

Limited expressivity but enables **efficient inference**. Can only derive positive information.

Definite Clauses

A **definite clause** is a Horn clause with **exactly one positive literal**.
That positive literal is the **head**, and the others are the **body**.

Form: $B \leftarrow A_1 \land A_2 \land \ldots \land A_n$

or equivalently $A_1 \land A_2 \land \ldots \land A_n \rightarrow B$

🧠 **Example:**

$Rain \land Cloudy \rightarrow Wet$

→ This is a **definite clause** because it has one positive literal (Wet).

A **definite clause** has the form:

$$(p_1 \land \cdots \land p_k) \rightarrow q$$

Valid Examples

- (Rain ∧ Snow) → Traffic
- Traffic (standalone)

Invalid Examples

- Rain ∧ Snow (no implication)
- ¬Traffic (negation)
- (Rain ∧ Snow) → (Traffic ∨ Peaceful)

Horn Clauses

A **Horn clause** is a logical statement **(disjunction of literals)** that has **at most one positive literal** (non-negated one).

Form: $(\neg A_1 \vee \neg A_2 \vee \ldots \vee \neg A_n \vee B)$

Equivalent in implication form: $(A_1 \wedge A_2 \wedge \ldots \wedge A_n) \rightarrow B$

🧠 **Example:** $Rain \wedge Cloudy \rightarrow Wet$

- In disjunction form: $\neg Rain \vee \neg Cloudy \vee Wet$

✅ **Why special:**
Horn clauses make **reasoning (inference) very efficient**.
They are the foundation of **logic programming** (like *Prolog*).

A **Horn clause** = a logical rule with at most one conclusion.
It is also either:

- A **definite clause:** $(p_1 \wedge \cdots \wedge p_k) \rightarrow q$
- a **goal clause:** $(p_1 \wedge \cdots \wedge p_k) \rightarrow false$

Valid Examples

- (Rain ∧ Snow) → Traffic
- (Traffic ∧ Accident) → false
- equivalent to: ¬ (Traffic ∧ Accident)



Completeness Comparison

Horn Clauses + Modus Ponens

Complete ✓

Efficient and sufficient for restricted formulas

General Logic + Modus Ponens

Incomplete □

Modus ponens alone is insufficient

General Logic + Resolution

Complete ✓

Powerful enough for all propositional logic

Modus Ponens in Action

Inference rule:

$$\frac{p_1, \cdots, p_k, \quad (p_1 \wedge \cdots \wedge p_k) \rightarrow q}{q}$$

Example Application

From Rain and Snow and (Rain $\wedge$ Snow $\rightarrow$ Traffic)

Derive: Traffic



Modus Ponens in Practice

The **modus ponens inference rule** allows us to derive **new conclusions** from existing knowledge:

$$\frac{p_1, \cdots, p_k, (p_1 \wedge \cdots \wedge p_k) \rightarrow q}{q}$$

Example

$$\frac{Wet, Weekday, Wet \wedge Weekday \rightarrow Traffic}{Traffic}$$

This rule captures the intuitive notion:

if we know all the premises are true, and we know that those premises imply a conclusion, then we can derive that conclusion.

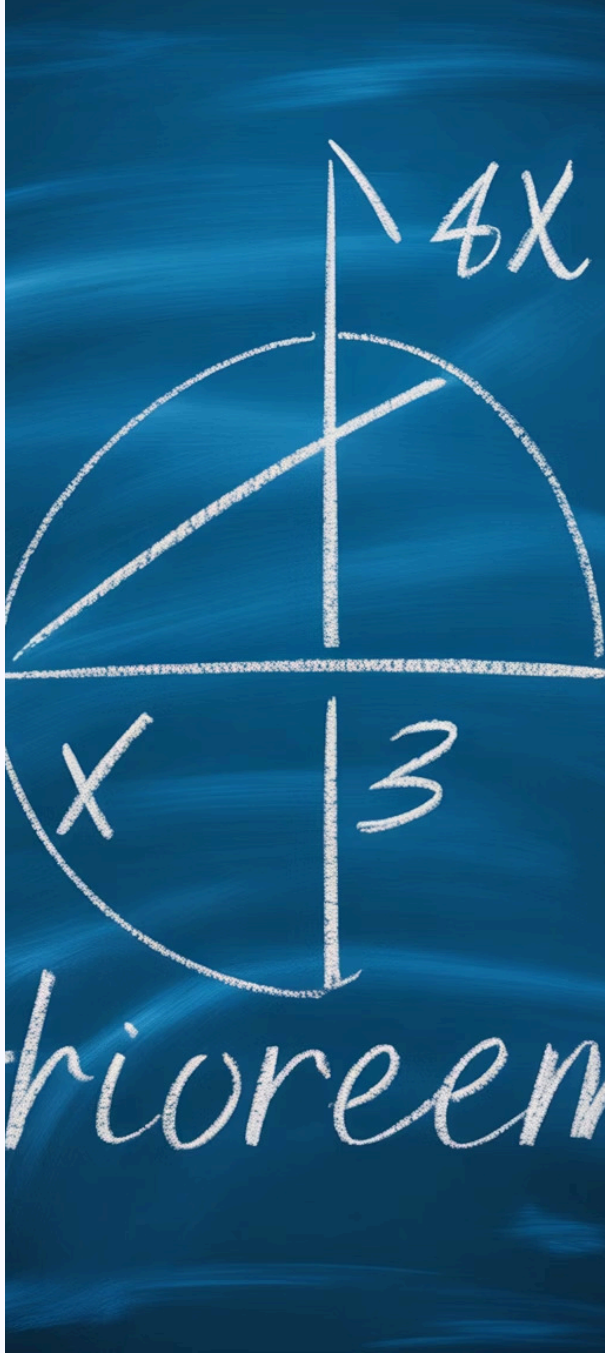
Completeness of Modus Ponens

Theorem: Modus Ponens on Horn Clauses

Modus ponens is complete with respect to Horn clauses.

Theorem: If **KB** contains only Horn clauses and ***p*** is entailed, then applying modus ponens will derive ***p***

Upshot: **KB** ⊨ **p** (entailment) equals **KB** ⊢ **p** (derivation)



Answering Queries

Knowledge Base

- Rain
- Rain → Wet
- Wet → Traffic

Query: Ask[Traffic]

"Yes" case: KB Traffic

Check: KB ∪ {Traffic → false} false?

"No" case: KB ¬Traffic

Problem: KB ¬Traffic impossible with Horn clauses!

Proof By Contradiction

Query: Ask[Traffic]

⚙️ **Goal:** Prove that from

$KB = \{Rain, Rain \rightarrow Wet, Wet \rightarrow Traffic\}$

we can conclude **Traffic** by **contradiction**.

🧠 Step 1: Assume the opposite (¬Traffic)

We temporarily add the **negation of what we want to prove**:

$KB \cup \{\neg Traffic\}$

Now the knowledge base becomes:

$\{Rain, Rain \rightarrow Wet, Wet \rightarrow Traffic, \neg Traffic\}$

🧩 Step 2: Apply inference rules

1. From **Rain** and **Rain → Wet**, **derive** **Wet**.
   $Rain, (Rain \rightarrow Wet) \vdash Wet$
2. From **Wet** and **Wet → Traffic**, **derive** **Traffic**.
   $Wet, (Wet \rightarrow Traffic) \vdash Traffic$
3. But we also have **¬Traffic** in the KB.

⚠️ Step 3: Derive a Contradiction

Now both **Traffic** and **¬Traffic** are true – impossible!

$Traffic \wedge \neg Traffic$

That means our assumption (**¬Traffic**) was wrong.

✅ Step 4: Conclude

Therefore, by **proof by contradiction**:

$KB \vDash Traffic$

> 💬 **In simple words:** We assumed there's *no traffic*, but that assumption led logically to *traffic exists*, so it must be false – hence **traffic must be true**.



Transition to Full Propositional Logic

From Horn Clauses to Propositional Full Logic

Horn clauses + modus ponens work perfectly together

But **general propositional logic** needs more power

Enter: Resolution

Resolution in Propositional Logic

Moving beyond Horn clauses to handle the full expressiveness of propositional logic requires a more powerful inference mechanism.

From Implication to Disjunction

Horn clauses can be equivalently written using disjunction instead of implication:

1

Implication Form

$A \rightarrow C$

$A \wedge B \rightarrow C$

2

Disjunction Form

$\neg A \vee C$

$\neg A \vee \neg B \vee C$

Literal

Either p or $\neg p$, where p is a propositional symbol

Clause

Disjunction of literals

Horn Clause

At most one positive literal

Modus ponens rewritten: From A and $\neg A \vee C$, derive C. Intuition: cancel out A and $\neg A$

From Implication to Disjunction

Horn clauses can be equivalently written using disjunction instead of implication:

Implication Form	Disjunction Form
$A \to C$	$\neg A \vee C$
$A \wedge B \to C$	$\neg A \vee \neg B \vee C$

Literal: either a proposition p or its negation ¬p (propositional symbol)

Clause: disjunction (OR) of literals

Horn clause: at most **one positive literal (e.g.** $\neg A \vee \neg B \vee C$**)**

Cancellation Intuition

Modus Ponens (rewritten)

$$\frac{A, \quad \neg A \vee C}{C}$$

Intuition: **A** and **¬A** cancel out

⚠ But this only handles one positive literal

What about general clauses with **multiple** positive literals?

Resolution: The Power Rule

Definition: Resolution is a single, **general inference rule** used to derive new clauses by **eliminating complementary literals** (one positive, one negative).

General clauses can have any number of literals:

$$\neg A \vee B \vee \neg C \vee D \vee \neg E \vee F$$

Key Features:

- **Sound** (derives only true consequences).
- **Complete** for propositional logic.
- Operates on CNF (Conjunctive Normal Form).

Key Features:

- **Sound** (derives only true consequences).
- **Complete** for propositional logic.
- Operates on CNF (Conjunctive Normal Form).

Resolution Inference Rule

$$\frac{f_1 \vee \cdots \vee f_n \vee \mathbf{p}, \quad \neg\mathbf{p} \vee g_1 \vee \cdots \vee g_m}{f_1 \vee \cdots \vee f_n \vee g_1 \vee \cdots \vee g_m}$$

→ Cancels out **p** and **¬p**

Intuition

- Works purely on **syntactic matching** (symbols, not meaning).
- Combines clauses to shrink the search space.
- Fundamental rule for **automated theorem proving**.

Example

$$\frac{\text{Rain} \vee \text{Snow}, \quad \neg\text{Snow} \vee \text{Traffic}}{\text{Rain} \vee \text{Traffic}}$$

Verifying Soundness of Resolution

Consider the example:

$$\frac{\text{Rain} \vee \text{Snow}, \quad \neg\text{Snow} \vee \text{Traffic}}{\text{Rain} \vee \text{Traffic}}$$

To verify soundness, we check the semantic interpretation:

M(Rain ∨ Snow) ∩ M(¬Snow ∨ Traffic) ⊆ M(Rain ∨ Traffic)?

Verifying Soundness of Resolution

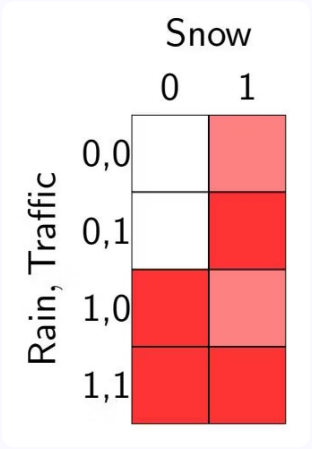
Consider the example:

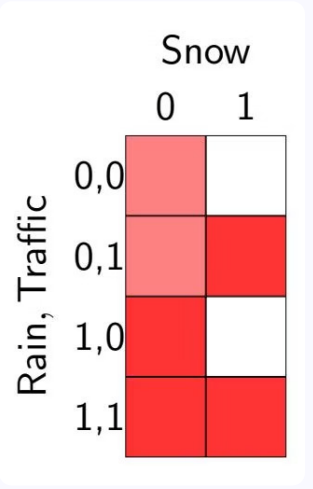
$$\frac{\text{Rain} \vee \text{Snow}, \quad \neg\text{Snow} \vee \text{Traffic}}{\text{Rain} \vee \text{Traffic}}$$

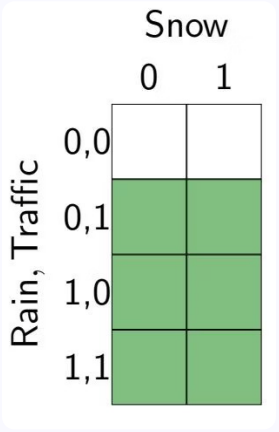
To verify soundness, we check the semantic interpretation:

$M(Rain \vee Snow) \cap M(\neg Snow \vee Traffic) \subseteq M(Rain \vee Traffic)$?

M(Rain ∨ Snow) ∩ M(¬Snow ∨ Traffic) ⊆ M(Rain ∨ Traffic)?







Sound! ✓

Conjunctive normal form

Conjunctive Normal Form (CNF)

Idea: Resolution works only on **clauses**, but that's fine–because *any* propositional formula can be rewritten in CNF.

Definition: Conjunctive Normal Form (CNF)

A **CNF formula** is a **conjunction (AND)** of **clauses**, and each **clause** is a **disjunction (OR)** of **literals**.

Example: $(A \vee B \vee \neg C) \wedge (\neg B \vee D)$

Interpretation: A CNF represents a **knowledge base**, where each clause is a separate formula (fact or rule).

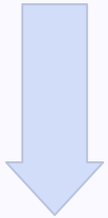
Key Proposition: ✅ *Same models → Same meaning*

Every formula f in propositional logic can be converted into an equivalent CNF formula f' where:

$$M(f) = M(f')$$

CNF Conversion Example

Let's walk through an example to convert a propositional formula into Conjunctive Normal Form (CNF) step-by-step.



Initial Formula

$$(Summer \rightarrow Snow) \rightarrow Bizarre$$



1. Remove Implication (A → B ≡ ¬A ∨ B)

$$\neg(\neg Summer \vee Snow) \vee Bizarre$$



2. Push Negation Inwards (De Morgan's Laws)

$$(\neg\neg Summer \wedge \neg Snow) \vee Bizarre$$



3. Remove Double Negation (¬¬A ≡ A)

$$(Summer \wedge \neg Snow) \vee Bizarre$$



4. Distribute ∨ over ∧ (A ∨ (B ∧ C) ≡ (A ∨ B) ∧ (A ∨ C))

$$(Summer \vee Bizarre) \wedge (\neg Snow \vee Bizarre)$$

The final result is in CNF: a conjunction of two clauses, where each clause is a disjunction of literals.

Complete CNF Conversion Rules

To systematically convert any propositional formula into **Conjunctive Normal Form (CNF)**, we apply these six fundamental rewrite rules:

01

Eliminate Biconditional

$$f \leftrightarrow g \quad \equiv \quad (f \rightarrow g) \wedge (g \rightarrow f)$$

Replaces biconditionals with conjunctions of implications, simplifying complex equivalences.

02

Eliminate Implication

$$f \rightarrow g \quad \equiv \quad \neg f \vee g$$

Converts all implications into disjunctions, moving closer to the CNF structure.

03

Move ¬ Inwards ($\wedge$)

$$\neg(f \wedge g) \quad \equiv \quad \neg f \vee \neg g$$

Applies De Morgan's Law to push negations into conjunctions.

04

Move ¬ Inwards ($\vee$)

$$\neg(f \vee g) \quad \equiv \quad \neg f \wedge \neg g$$

Applies De Morgan's Law to push negations into disjunctions.

05

Double Negation

$$\neg\neg f \quad \equiv \quad f$$

Removes redundant negations, simplifying expressions.

06

Distribute $\vee$ over $\wedge$

$$f \vee (g \wedge h) \quad \equiv \quad (f \vee g) \wedge (f \vee h)$$

Distributes disjunctions over conjunctions to form clauses composed of disjunctions.

Each operation above preserves the semantic meaning of the formula. This systematic approach ensures any propositional formula can be transformed into an equivalent CNF, facilitating inference through resolution.

Resolution Algorithm Steps

🧠 **Recall:** Entailment and contradiction are both linked to **satisfiability**:

- $KB \models f \Leftrightarrow KB \cup \{\neg f\}$ is **unsatisfiable**
- $KB \models \neg f \Leftrightarrow KB \cup \{f\}$ is **unsatisfiable**

Key Insight:
Resolution works by *refuting* the negation of the goal if contradiction is found, the original statement is **entailed** ✅

⚙️ Resolution-Based Inference

Algorithm Steps:

1. **Convert** all formulas into **CNF (Conjunctive Normal Form)**
2. **Apply** the **resolution rule** repeatedly
3. **Stop** when either:
   - You derive **false** → **KB ⊢ f** (entailment confirmed)
   - No new clauses can be derived → satisfiable (no contradiction)

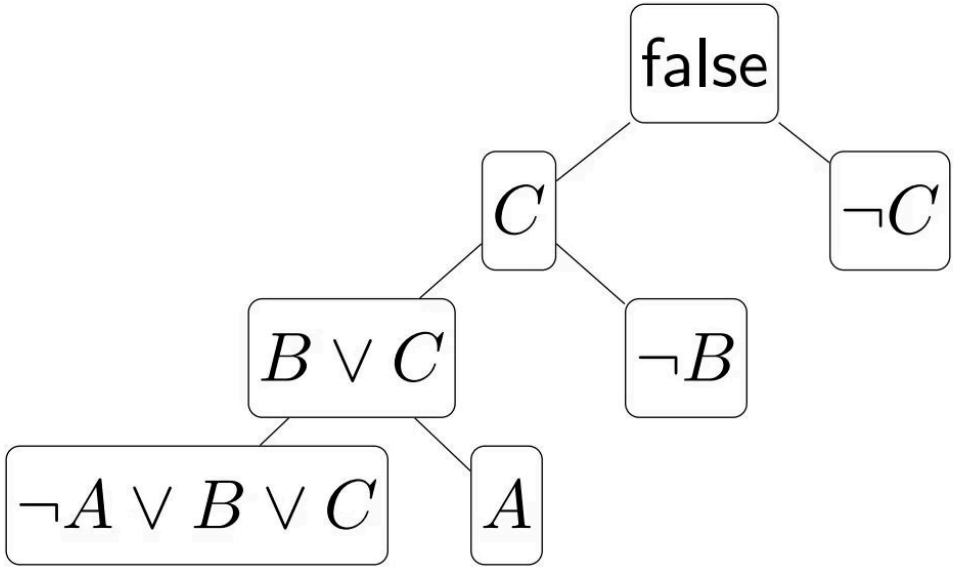
Resolution Example: Step by Step

Knowledge Base: $A \rightarrow (B \vee C), A, \neg B, \neg C$

Convert to CNF: $\neg A \vee B \vee C, A, \neg B, \neg C$

1. Resolve A and ¬A ∨ B ∨ C → get B ∨ C
2. Resolve B ∨ C and ¬B → get C
3. Resolve C and ¬C → get **false**

Conclusion: KB is unsatisfiable!



The Cost of Expressiveness

Modus Ponens

$$\frac{p_1, \cdots, p_k, (p_1 \wedge \cdots \wedge p_k) \rightarrow q}{q}$$

Adds clause with **one** symbol

Linear time Complexity

Resolution

$$\frac{f_1 \vee \cdots \vee f_n \vee p, \quad \neg p \vee g_1 \vee \cdots \vee g_m}{f_1 \vee \cdots \vee f_n \vee g_1 \vee \cdots \vee g_m}$$

Each rule application adds clauses with many propositional symbols

Exponential time complexity

The Expressivity-Efficiency Trade-off



Horn Clauses

Inference: Modus Ponens

Complexity: Linear time

Expressivity: Limited

Efficient but restricted in what can be expressed.



General Clauses

Inference: Resolution

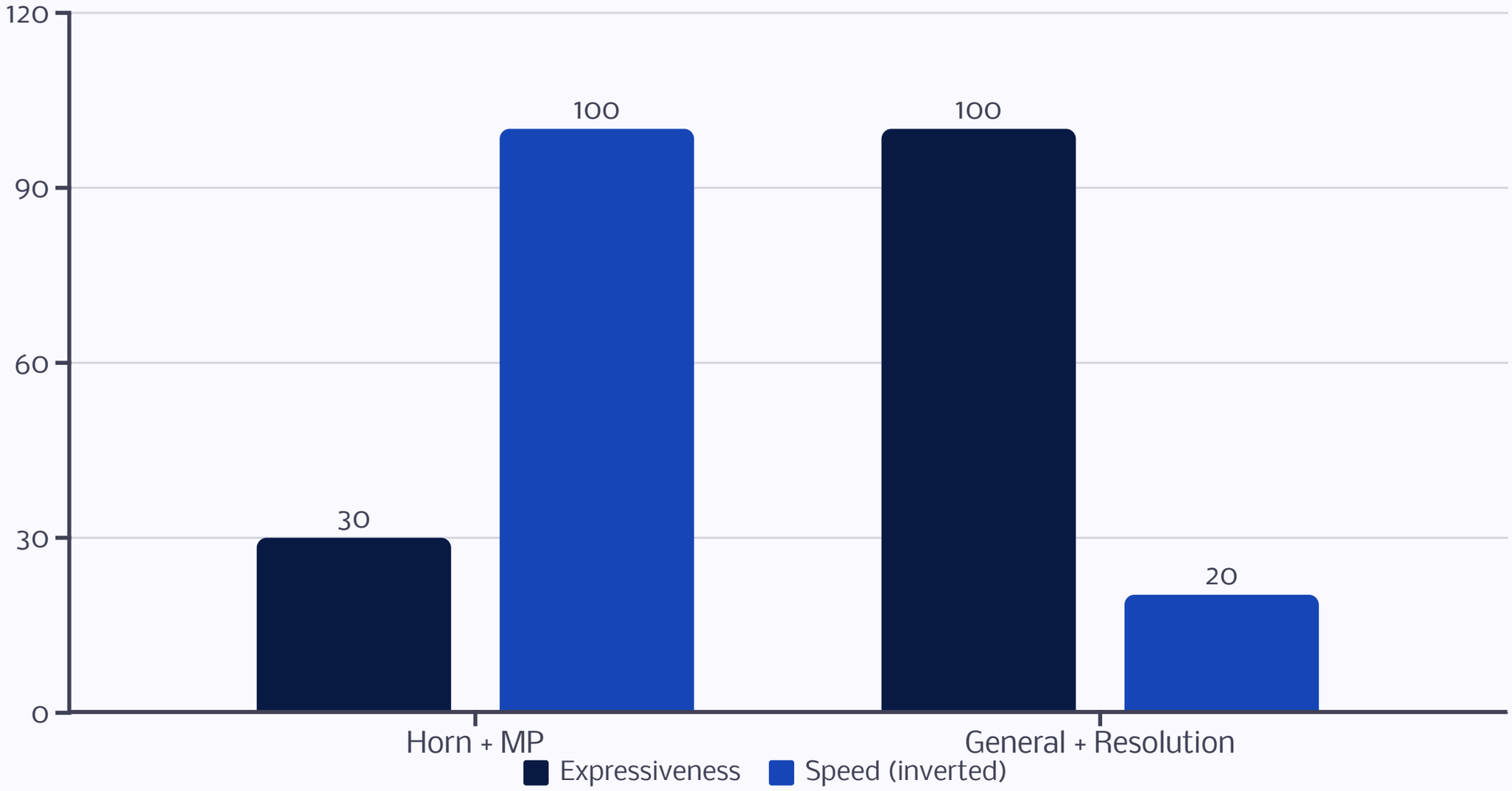
Complexity: Exponential time

Expressivity: Full propositional logic

Powerful but computationally expensive.

The choice between these approaches depends on your specific needs: prioritize efficiency with limited expressivity, or opt for full expressivity at a higher computational cost.

The Expressiveness-Efficiency Tradeoff



Choose based on your needs: **efficiency** or expressiveness

Limitations of Propositional Logic

While powerful for certain logical inferences, propositional logic encounters significant challenges when attempting to model more complex scenarios:

Specific Facts

"Alice and Bob both know arithmetic."

$$AliceKnowsArithmetic \land BobKnowsArithmetic$$

Feasible, but quickly becomes unwieldy for a larger number of specific individuals or objects.

General Statements

"All students know arithmetic."

$$AliceIsStudent \rightarrow AliceKnowsArithmetic$$

$$BobIsStudent \rightarrow BobKnowsArithmetic$$

...

Expressing universal quantification requires a separate formula for each instance, making it impractical for large or infinite domains.

Complex Relationships

"Every even integer greater than 2 is the sum of two primes."

???

Statements involving quantification over variables (e.g., "every," "some") **cannot be finitely and accurately expressed.**

What's Missing?

While propositional logic is effective for simple truth statements, it struggles with the inherent structure and generalization found in everyday knowledge:

Objects & Relations

Propositions like `AliceKnowsArithmetic` have an internal structure that we're ignoring: **Alice** (object), **Knows** (relation), **Arithmetic** (object).

Quantifiers & Variables

The word **"all"** is a quantifier that applies to each person. We don't want to enumerate them all individually–we need variables to represent arbitrary individuals and general rules.



What's Missing?



Objects & Relations

Propositions like `AliceKnowsArithmetic` have internal structure that we're ignoring:

alice (object), Knows (relation), arithmetic (object)



Quantifiers & Variables

Universal (∀) and existential (∃) quantifiers

$\forall x.\ \text{Student}(x) \rightarrow \text{KnowsArithmetic}(x)$

The Power of Quantification

Propositional Logic (P.L.)

In propositional logic, expressing a general statement like **"All students know arithmetic"** requires enumerating each individual case:

$$AliceIsStudent \rightarrow AliceKnowsArithmetic$$

$$BobIsStudent \rightarrow BobKnowsArithmetic$$

$$CharlieIsStudent \rightarrow CharlieKnowsArithmetic$$

This quickly becomes **unmanageable** and **impractical** for large or infinite domains.

First-Order Logic (F.O.L.)

First-Order Logic elegantly solves this by introducing **quantifiers** and **variables**, allowing a single formula to express universal truths:

$$\forall x.Student(x) \rightarrow KnowsArithmetic(x)$$

This single formula concisely captures the rule for **all** individuals, offering immense **expressive power** and **scalability**.

Impossible in Propositional Logic

Goldbach's Conjecture

"Every even integer greater than 2 is the sum of two primes"

- This statement quantifies over **infinitely many** integers
- Cannot be expressed with finite propositional formulas
- Requires the power of first-order logic



Impossible in Propositional Logic

> "Every even integer greater than 2 is the sum of two primes."
>
> **— Goldbach's Conjecture**

This unsolved mathematical problem highlights propositional logic's inherent limitations for complex mathematical statements:

→ Infinite Domains & Quantifiers

The conjecture involves an infinite set of numbers. Propositional logic lacks the variables and quantifiers needed to express universal statements about these infinite domains.

→ No Internal Structure Analysis

Propositional logic treats statements as atomic facts. It cannot analyze the internal structure of mathematical concepts like "even integer" or "prime" to establish relationships.

→ Beyond Atomic Truths

It is confined to truth values of simple propositions, unable to represent or reason about complex mathematical structures or their properties.



Introducing First-Order Logic

To overcome the expressivity limitations of propositional logic, particularly in handling objects, relations, and quantification, we turn to a more powerful formalism: **First-Order Logic (FOL)**.

FOL extends propositional logic with crucial features that enable a richer representation of knowledge and more sophisticated reasoning.

Objects & Predicates

Allows us to represent individual objects (e.g., Alice, student) and their properties or relationships (e.g., $KnowsArithmetic(Alice)$, $Student(x)$).

Quantifiers & Variables

Introduces universal ($\forall$) and existential ($\exists$) quantifiers to express general statements about collections of objects.

These additions empower FOL to model complex real-world scenarios and mathematical theories with unparalleled precision and expressiveness.

Key Takeaways

1

Resolution is Complete

Works for all propositional logic, but at exponential cost

2

CNF is Universal

Every formula can be converted to conjunctive normal form

3

Tradeoffs Matter

Horn clauses: efficient but limited; general logic: expressive but costly

4

Beyond Propositions

First-order logic adds objects, relations, and quantifiers

Looking Forward

What We Learned

- Resolution inference rule
- CNF conversion techniques
- Completeness guarantees
- Computational tradeoffs

What's Next

- First-order logic syntax
- Quantifiers and variables
- Unification algorithms
- Practical applications

Questions?

